

第 14 组 乐赛系统 软件设计文档

1. 任务分工

任务	完成人
1、软件体系结构设计修订与汇总	張振豐、陈俊皓
2、用户界面设计修订与截图更新	張振豐、李峻安
3、软件详细设计与数据库设计修订	張振豐、董祉含、路良钧、张二思
4、最终文档整合与格式校对	張振豐

2. 文档概述

2.1 文档目的

本文档为乐赛系统的软件设计文档，用于汇总并修订前期软件体系结构设计、用户界面设计和软件详细设计。文档内容以当前最终系统为准，重点描述系统的分层架构、部署结构、用户界面职责、界面跳转关系、核心业务详细设计、数据对象设计和与前期设计相比的主要变更。

2.2 设计范围

本文档覆盖 Web 前端、Django 后端、MySQL 数据库、静态资源与上传资源等主要组成部分，覆盖登录注册、赛事大厅、赛事详情、创建赛事、赛事工作台、报名管理、单淘汰树、好友系统、消息通知和管理员审核等已实现功能。

2.3 设计约束

- 系统采用前后端分离架构，前端通过 Axios 调用后端 REST 风格接口。
- 后端采用 Django 会话机制维护登录状态，并通过角色与资源归属控制权限。
- 赛事缩图、登录背景图等静态资源由前端 public 目录提供，用户上传缩图由后端媒体目录保存。
- 淘汰树不单独建复杂赛程表，当前通过 competition.bracket_state 保存节点状态、胜者和种子配置。

3. 软件体系结构设计

3.1 文档目的

本文档为乐赛系统的软件设计文档，用于汇总并修订前期软件体系结构设计、用户界面设计和软件详细设计。文档内容以当前最终系统为准，重点描述系统的分层架构、部署结构、用户界面职责、界面跳转关系、核心业务详细设计、数据对象设计和与前期设计相比的主要变更。

乐赛系统软件体系结构包图

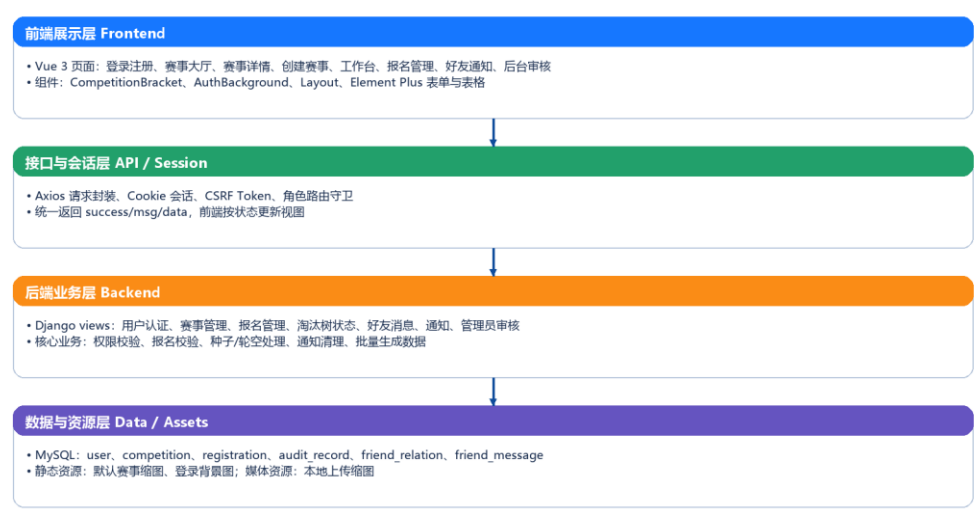


图 2-1 乐赛系统软件体系结构包图

3.2 分层职责

层次	主要组成	职责
前端展示层	Vue 3、Element Plus、Pinia、Vue Router	提供登录注册、赛事大厅、赛事详情、赛事创建、报名管理、淘汰树、好友通知和后台审核界面。
接口与会话层	Axios、Cookie、CSRF、路由守卫	封装请求、处理登录状态、拦截未授权访问，并把后端状态转换为页面反馈。
后端业务层	Django views、业务函数、权限判断	处理赛事、报名、审核、好友、通知、批量生成和淘汰树状态保存等业务逻辑。
数据与资源层	MySQL、public 静态资源、media 上传资源	持久化业务数据，提供默认缩图、登录背景和本地上传图片等资源。

3.3 重用软件资源与技术选型

资源/技术	使用位置	设计说明
Vue 3	前端页面与组件	用于构建响应式单页应用，按页面和业务组件拆分界面。
Vite	前端开发与构建	提供快速开发服务和构建能力。
Element Plus	前端交互控件	复用表单、按钮、表格、弹窗、标签、空状态等 UI 组件。
Axios	前后端通信	统一发送 API 请求并携带 Cookie 会话。
Django	后端 API 与会话	提供路由、视图函数、会话管理和数据库模型映射。
MySQL	数据持久化	保存用户、赛事、报名、审核、好友消息等核心业务数据。
Playwright	文档截图验证	用于获取当前系统真实界面截图，保证设计文档与最终界面一致。

3.4 部署架构描述

当前开发环境中，前端运行在 Vite 服务上，后端运行在 Django 开发服务上，二者通过 HTTP 接口通信。生产部署时，前端构建产物可由 Nginx 或同类静态资源服务器托管，后端作为独立 API 服务运行，并连接 MySQL 数据库与媒体资源目录。

乐赛系统部署架构图

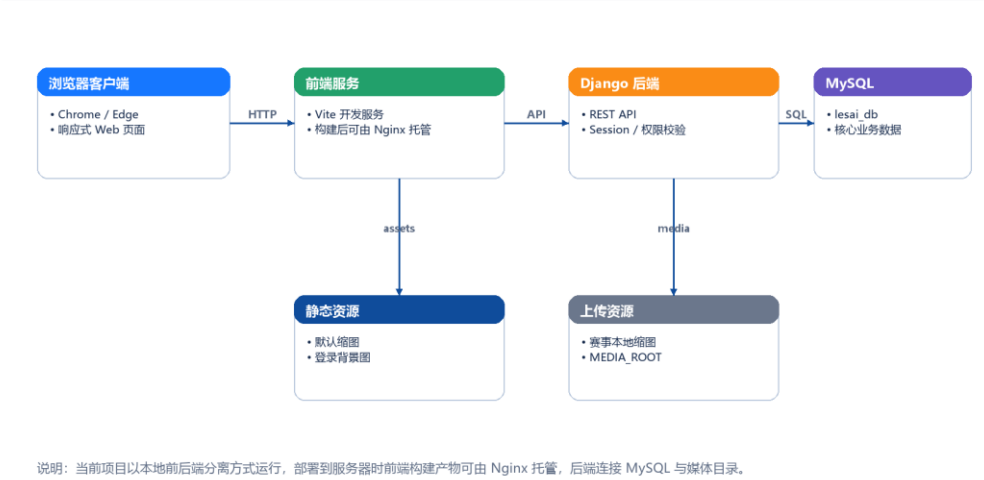


图 2-2 乐赛系统部署架构图

4. 用户界面设计

4.1 界面设计原则

最终系统界面已经由早期静态页面调整为完整的赛事管理型 Web 应用。整体采用左侧导航、右侧工作区的管理界面结构，登录与注册页面采用赛事图片背景，赛事大厅使用卡片式赛事列表，管理页面使用表格、筛选器、按钮组和可视化淘汰树，以保证演示时信息清晰、操作路径明确。

4.2 用户界面及其职责

界面名称	标识	职责
登录界面	LoginView	完成账号、密码、验证码登录，并进入主应用框架。
注册界面	RegisterView	创建用户账号并选择用户角色。
主框架界面	Layout	提供侧边栏导航、角色入口、退出登录和未读通知角标。
赛事大厅	HomeView	展示赛事统计、筛选条件、赛事卡片和收藏入口。
赛事详情	EventDetailView	展示赛事图片、编号、时间地点、规则、奖励、邀请码和报名入口。
创建/编辑赛事	CreateCompetitionView / CompetitionEditView	维护赛事基础信息、时间地点、类型、缩图、规则和奖励。
赛事工作台	WorkbenchView	集中展示主办方或管理员可管理的赛事并提供状态操作。

报名管理与淘汰树	RegistrationManageView	管理报名名单、批量加入/删人、设置种子、渲染淘汰树并录入胜者。
好友系统	FriendsView	搜索用户编号、发送好友申请、聊天和删除好友。
消息通知	NotificationsView	展示好友、赛事、报名等通知并支持即时刷新和处理。
审核与风控	AdminReviewView	管理员审核赛事、管理用户、查看测试数据和审核记录。

4.3 界面跳转关系

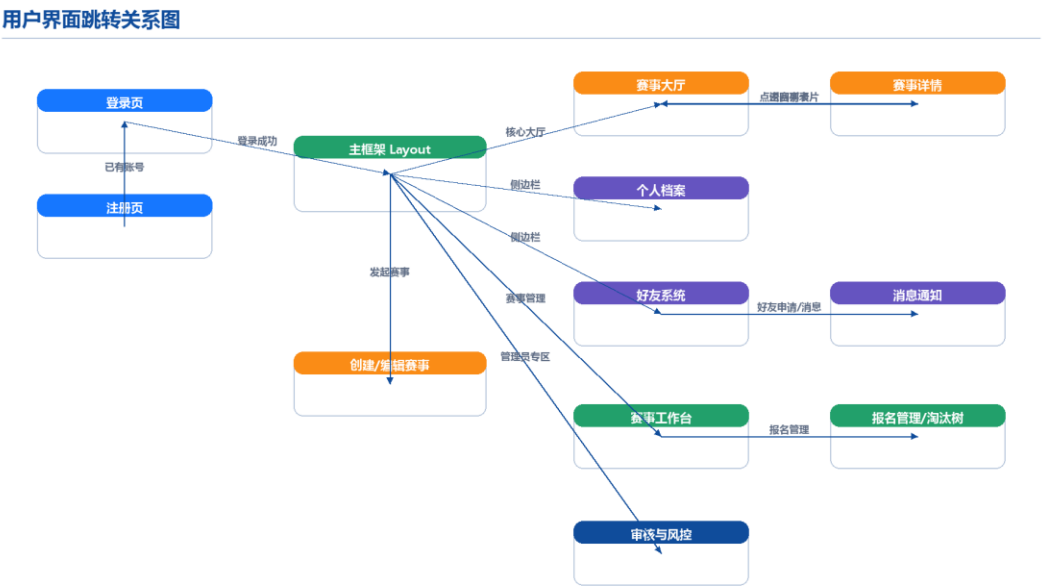


图 3-1 用户界面跳转关系图

4.4 当前系统主要界面

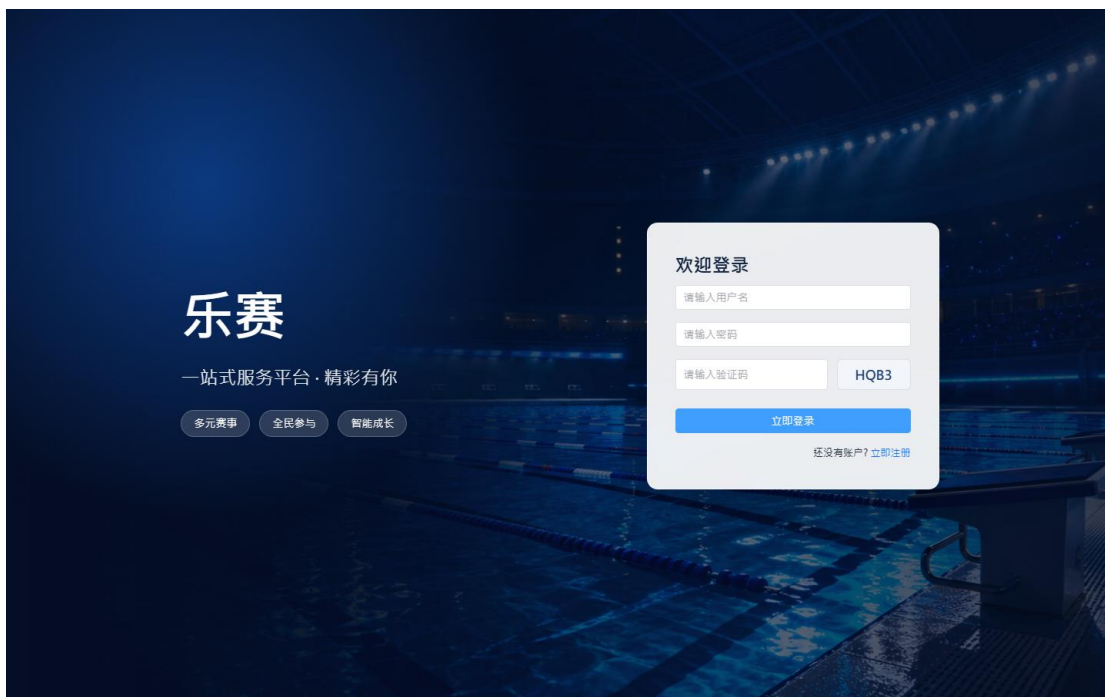


图 3-2 登录界面



图 3-3 注册界面



图 3-4 赛事大厅界面



图 3-5 赛事详情界面



图 3-6 创建赛事界面

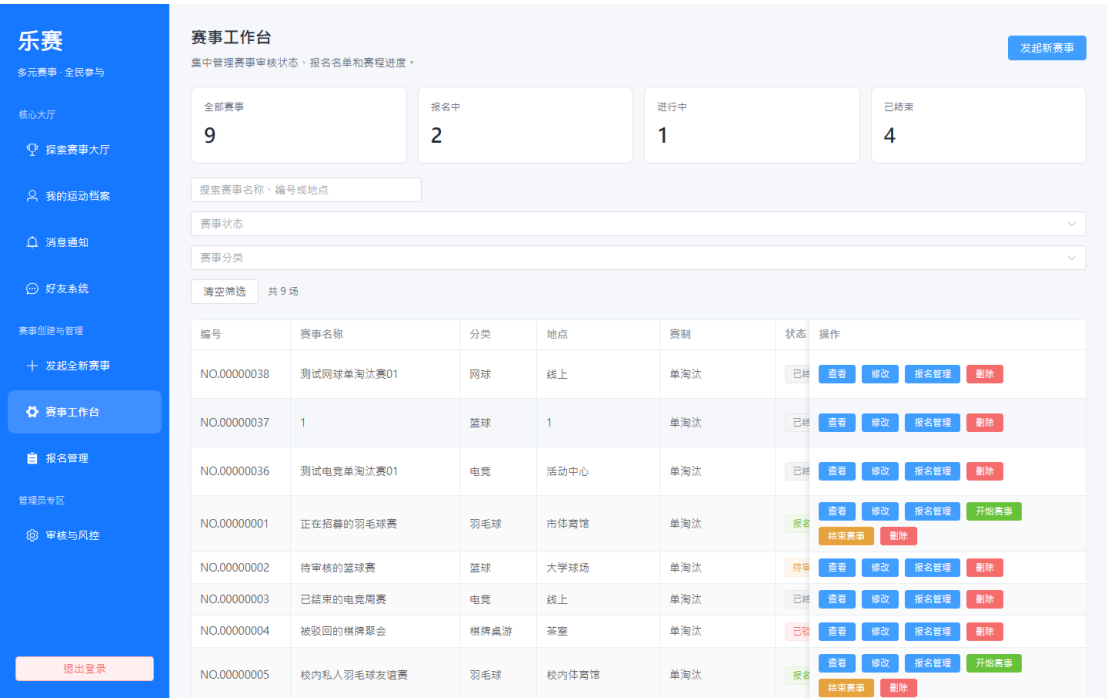


图 3-7 赛事工作台界面



图 3-8 报名管理与淘汰树界面

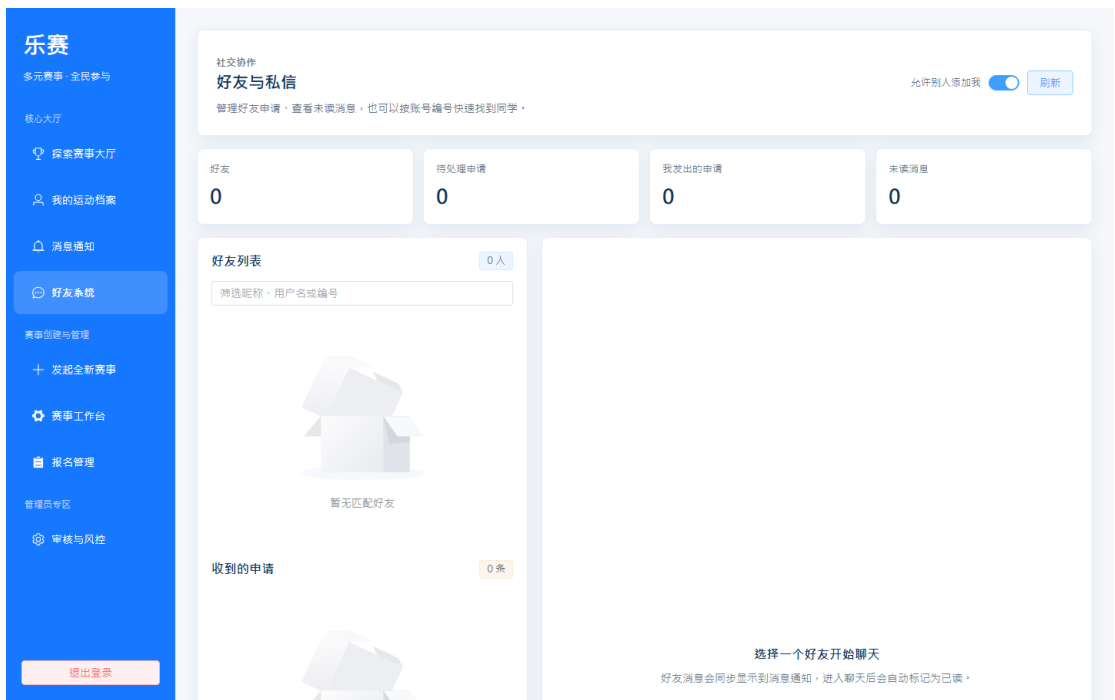


图 3-9 好友系统界面

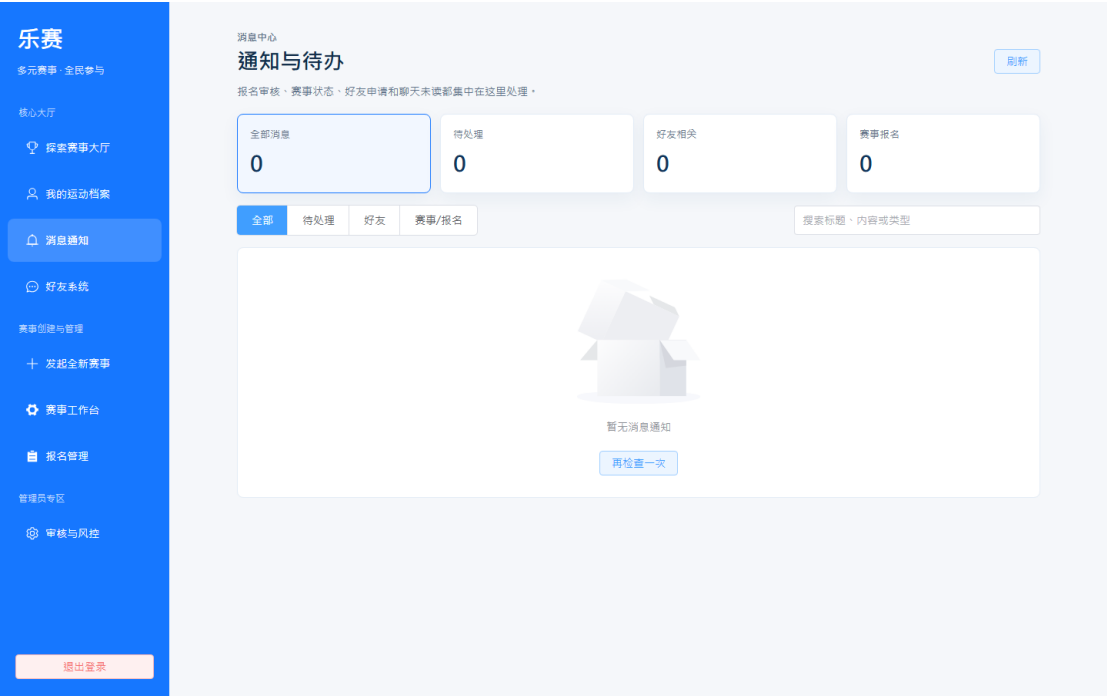


图 3-10 消息通知界面

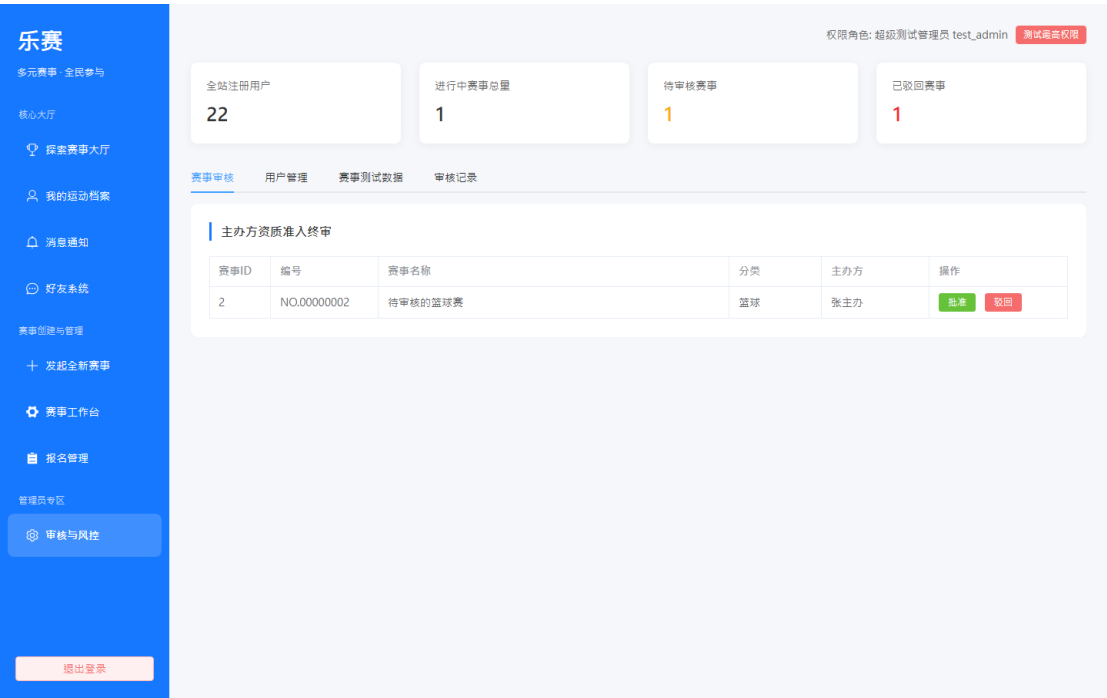


图 3-11 管理员审核与风控界面

5. 软件详细设计

5.1 详细设计范围

详细设计部分延续前期作业中以报名用例为核心的设计思路，但按照当前系统实现进行修订。当前系统的关键业务已经从单纯报名扩展为“报名管理 + 批量人员维护 + 淘汰树生成 + 胜者晋级 + 通知联动”的完整赛事流程，因此本节围绕该业务链路描述顺序图、设计类图、活动图和关键方法。

5.2 报名与淘汰树维护顺序图



图 4-1 报名与淘汰树维护详细顺序图

该顺序图体现了从用户提交报名、主办方维护报名名单，到生成淘汰树、保存 bracket_state、录入胜者并推进下一轮的核心交互。前端负责采集操作和刷新视图，Django API 负责权限校验与业务处理，MySQL 负责保存报名、赛事和通知状态，淘汰树组件负责根据报名数据渲染赛程。

5.3 设计类图

报名与赛事管理设计类图

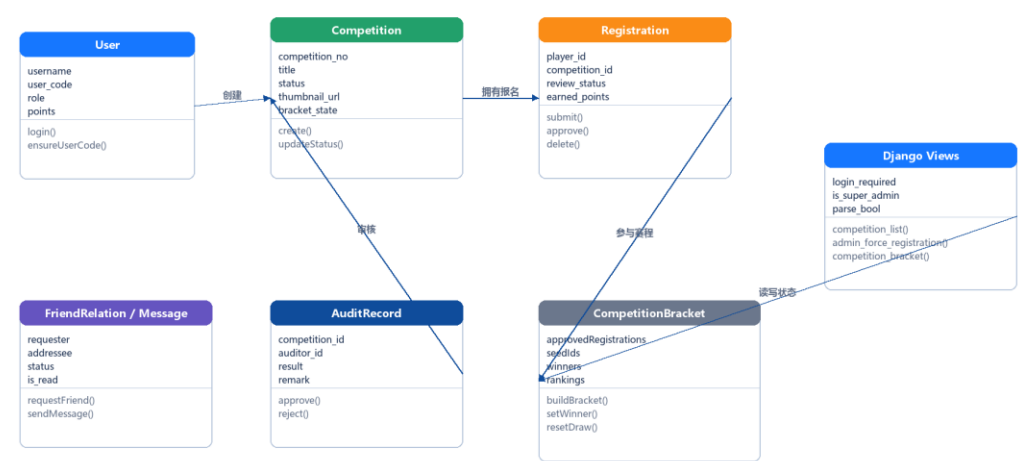


图 4-2 报名与赛事管理设计类图

当前系统的核心实体包括 User、Competition、Registration、AuditRecord、FriendRelation 和 FriendMessage。淘汰树作为前端组件和赛事状态共同维护的设计对象，通过 Competition.bracket_state 保存胜者、种子、排名等状态，避免在当前规模下引入过重的赛程表结构。

5.4 活动图

赛事报名与赛程活动图

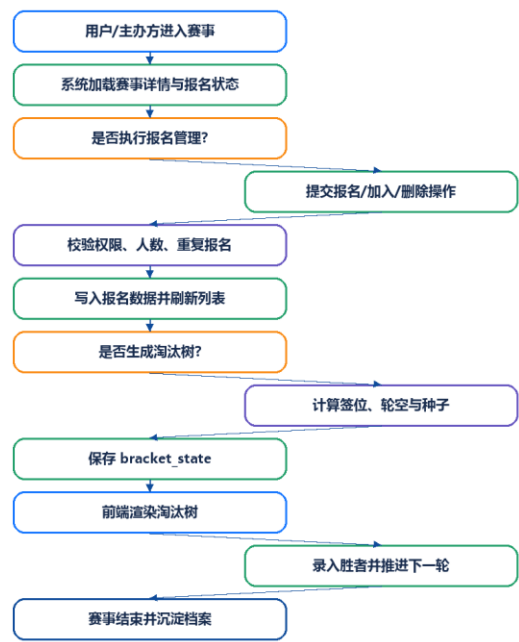


图 4-3 赛事报名与赛程活动图

活动图展示了用户进入赛事、系统加载报名状态、主办方执行报名管理、生成淘汰树、保存赛程状态、前端渲染树图以及录入胜者的完整处理路径。对于单人、无人、非 2 的幂人数和存在轮空位等边界情况，系统在生成淘汰树阶段进行分支处理。

5.5 核心方法说明

方法/接口	所属模块	设计说明
login()	用户认证	校验账号、密码、验证码或测试账号自动登录条件，写入 session。

competition_list()	赛事大厅	返回赛事列表、状态、人数、缩图和基础筛选数据。
create_competition()	赛事创建	保存赛事基础信息、赛事编号、缩图路径、时间地点和规则。
register_competition()	赛事报名	校验报名状态、人数上限、重复报名和私人邀请码后写入报名记录。
competition_registrations()	报名管理	返回赛事报名人员、审核状态、积分和当前 bracket_state。
admin_bulk_force_registration()	批量加入	由超级测试管理员批量生成或加入测试用户，并可生成随机积分。
competition_bracket()	淘汰树状态	保存或读取赛事淘汰树状态，包括胜者、种子模式、种子 ID 和排名。
send_friend_request()	好友系统	创建好友申请并生成通知。
notifications()	消息通知	返回当前用户通知与未读状态，支持即时刷新。
admin_users()	后台管理	返回用户列表、角色、编号、状态和统计数据。

6. 数据库与数据对象设计

6.1 主要数据表设计

表名	关键字段	设计说明
user	id, username, user_code, password, role, nickname, email, points, is_deleted, allow_friend_requests	保存平台用户、角色、编号、积分和好友申请设置。
competition	id, competition_no, title, category, location, type, organizer_id, status, max_participants, bracket_state, thumbnail_url	保存赛事基础信息、状态、主办方、缩图和淘汰树状态。
registration	id, player_id, competition_id, status, review_status, register_type, team_name, final_score, final_rank, earned_points	保存用户与赛事之间的报名关系和比赛结果。
audit_record	id, competition_id, auditor_id, result, remark, audit_time	保存管理员审核赛事的结果和备注。
friend_relation	id, requester_id, addressee_id, status, created_at, updated_at	保存好友申请与好友关系状态。
friend_message	id, sender_id, receiver_id, content, is_read, created_at	保存好友消息和已读状态。
point_history	id, username, change_amount, reason, time	保存积分变动历史。

6.2 数据一致性设计

- 报名成功或删除报名后，同步更新赛事当前报名人数。

- 报名管理中的批量删除会同步扣减人数，并避免留下无效报名记录。
- 删除好友时同步清理相关未读消息和通知，避免通知角标残留。
- 淘汰树状态与赛事绑定保存，刷新页面后仍能恢复胜者、种子和排名。
- 用户编号由后端生成并补齐，好友搜索统一支持用户名、昵称和用户编号。

7. 前期设计修订说明

7.1 体系结构设计修订

前期体系结构设计中曾包含 Spring Security、JJWT、Quartz、Knife4j、Druid、ShardingSphere、Redis、Apache POI 等设想。最终实现版本采用 Vue 3 + Django + MySQL 的轻量前后端分离架构，删除未实际落地的 Java 相关中间件描述，补充了会话认证、静态缩图、媒体上传、通知刷新和淘汰树状态保存等真实结构。

7.2 用户界面设计修订

前期用户界面设计以静态登录页和概念界面为主，最终系统已经形成真实可操作的赛事管理界面。本文档重新截取当前页面，替换旧 UI 草图，并补充赛事大厅、详情页、创建赛事、工作台、报名管理、淘汰树、好友、通知和后台审核等实际界面。

7.3 软件详细设计修订

前期详细设计主要围绕报名用例展开，最终系统在此基础上增加了批量生成参赛者、随机积分、种子选手首轮轮空、单人参赛处理、胜者晋级、好友通知同步清理等实现细节。本文档不新增需求用例分析，而是在原有报名与赛事管理设计基础上完善具体类、方法、活动和数据状态说明。

7.4 总结

修订后的软件设计文档与当前系统功能和界面保持一致，能够作为最终提交中的设计依据。文档保留前期作业的章节风格和设计表达方式，同时将内容统一到最终实现版本，避免旧技术栈、旧界面和旧数据结构与实际系统不一致。